

What is a GenAI framework?



Gorti Sai Brahmanandam

Follow

4 min read · 18 hours ago



28



A brief intro of...

What is a GenAI framework?

Of late, if any Software engineer, DevOps engineer, or SRE sees any job postings in the Tech area, one common jargon term they hit first is — AI.

Everyone knows what AI is, but some engineers are confused with the terminology used and have no idea where to start with it. Of course, our Tech Daddy Google is their ultimate help to reach out. Tech Generous Daddy dumps so many links and greets us — with so much of information.

The AI journey starts here, and the real confusion follows and builds up exponentially.

My main intention here is to give an intro dive into AI and what it could do to improve our tech lives.

First and foremost, we need to understand the meanings of the terminology used and the concepts of the **GenAI framework**.

So, What does — GenAI framework — provide?

A *GenAI framework* is a software toolkit that provides pre-built abstractions, components, and orchestration patterns for building applications powered by generative AI models — so developers don't have to wire together model APIs, memory, tool-calling, prompt management, and agent loops from scratch.

What does a *GenAI framework* give you?

In my previous article on “**Bot Management**”, I have mentioned about Bot operating methods:

Open in app ↗

Sign up

Sign in

Medium

Search

Write



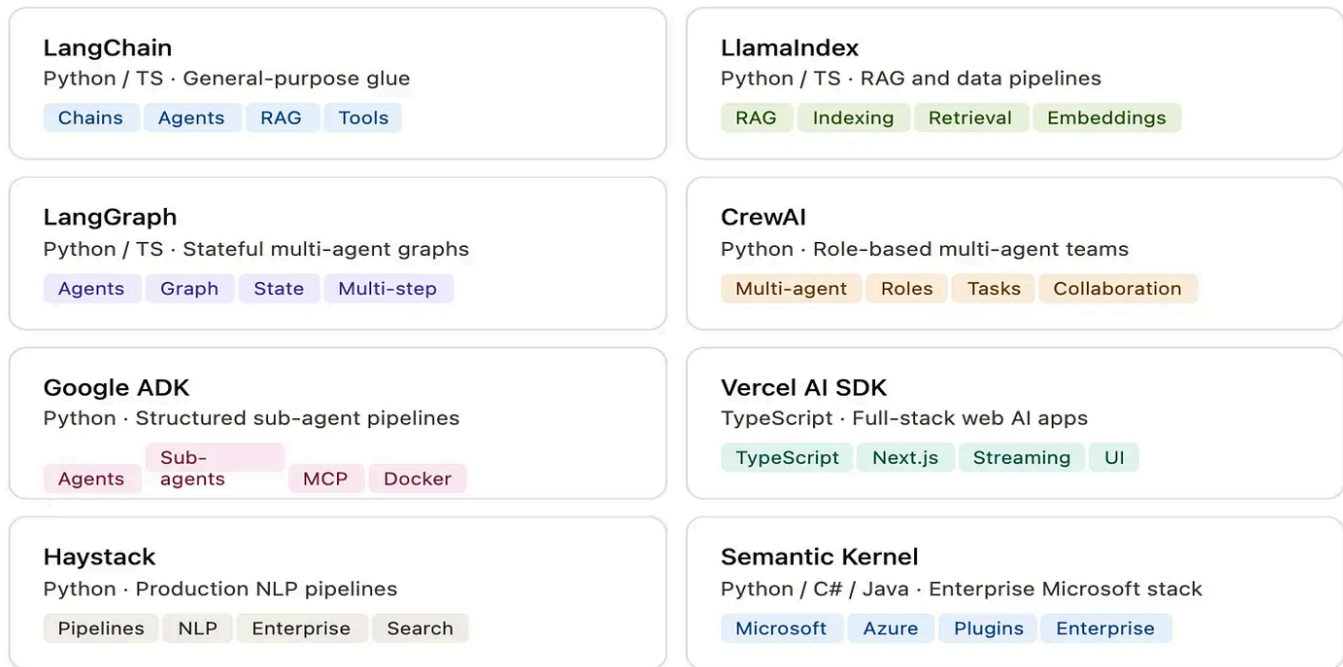
provide structure that meets the end goals.

Without a framework, building a simple RAG chatbot means you have to manually call the model API, manage conversation history, chunk documents, generate embeddings, run vector searches, inject retrieved context into prompts, handle token limits, and log everything — hundreds of lines of plumbing before a single line of actual application logic.

A GenAI framework handles all of that plumbing so you write only what's unique to your use case. Think of it like *Express* for web servers or *Spring* for

Java backends — you're building on proven abstractions rather than raw sockets.

Here are some of the frameworks used by many, depending on their engineering setup and requirements.



The five core components every GenAI framework provides:

1. **Prompt management** — templates with variable injection, system/human/assistant role separation, automatic truncation when context limits are approached, and conversation history compression. Without this you're doing string formatting by hand and constantly hitting token limit errors.

2. **Memory** — the mechanism that gives an *LLM continuity* across turns, since *models* themselves are *stateless*:

Type	How it works	Best for
Buffer memory	Append every turn verbatim	Short conversations
Summary memory	Periodically summarize old turns	Long conversations
Entity memory	Extract and store named entities	Customer support
Vector memory	Embed and retrieve relevant past turns	Long-term personalization
External memory	Store in Redis/Postgres, retrieve by similarity	Production systems

3. Tool / function calling — a standardized way for the model to invoke external capabilities (search, code execution, database queries, API calls) and observe the result before generating a final response. All frameworks wrap the model’s native function-calling protocol first, so tools are defined once and work across model providers.

4. Chains and graphs — composition primitives that connect multiple LLM calls, tool invocations, and conditional branches into a coherent workflow.

— **LangChain** calls these chains;

Get Gorti Sai Brahmanandam’s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

— **LangGraph** calls them graphs;

— **CrewAI** calls them crews;

— Google ADK calls them sequential/parallel agents.

It's like different cups; same coffee — *Different names, same underlying idea.*

5. Retrieval (RAG) — connectors to vector stores (Pinecone, Chroma, pgvector, Weaviate), document loaders (PDF, Confluence, Notion, S3), chunking strategies, and embedding model abstraction — so the same retrieval pipeline works whether you switch from OpenAI embeddings to local `omic-embed-text`.

I will provide — with some coding examples — how these frameworks differ from raw API calls.

The intention here is observe how the frameworks address similar queries based on the design, in their own way.

First, I will start with a sample code and provide details about how each framework differs from *raw API calls*.

The intention here is observe how the frameworks address similar queries based on the design, in their own way.

First, I will start with a sample code and provide details about how each framework differs from *raw API calls*.

Without a framkework:

```
# Raw — every project reinvents this
import openai, json
```

```
messages = [{"role": "system", "content": system_prompt}]
messages.append({"role": "user", "content": user_input})

response = openai.chat.completions.create(
    model="gpt-4o-mini", messages=messages)

reply = response.choices[0].message.content
messages.append({"role": "assistant", "content": reply})

# And here, I have only handled one turn, with no tools, no retrieval,
# no streaming, no error handling, no token counting and so on
```

With *Langchain* framework:

```
from langchain.chat_models import ChatOpenAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_community.tools import DuckDuckGoSearchRun

llm = ChatOpenAI(model="gpt-4o-mini")
tools = [DuckDuckGoSearchRun()]

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools)
executor.invoke({"input": "Find recent outages for stripe.com"})

# Memory, tool dispatch, retry, streaming – all handled
```

Decision guidelines for the use case(s):

Given the DevOps / SRE mindset I have and the work I've been doing for a long time — and my previous articles on *EDA*, *RAG assistant*, *Docker Agentic apps*, *SRE tooling* — here's the practical or maybe suggested mapping.

The honest answer about which framework to choose is:

- **LangChain** for breadth,

- **LlamaIndex** for retrieval quality,
- **LangGraph** for complex agent state,
- **Vercel AI SDK** if you're in TypeScript.

Note: Once again, the decisions are primarily individual choices, depending on the work setup, the tools used, and the tech environment.

Of all, *LlamaIndex*, is popular and most production systems end up using it for the *retrieval layer* and *LangGraph* for the orchestration layer together — though they're complementary, not competing.

Finally — I hope this gives an idea of the GenAI framework and how anyone can explore and make a choice depending upon their requirements and the environment they work in.

— 00000—

[Gen Ai](#)[Bots](#)[Python Ai](#)

Written by Gorti Sai Brahmanandam

11 followers · 7 following

[Follow](#)

